

Додаток 1. Лістинг коду програми

apps\lab2.cpp

```
1  #include <windows.h>
2
3  #include <Args.hpp>
4  #include <DotTreeView.hpp>
5  #include <FileSource.hpp>
6  #include <IdentifiersTableView.hpp>
7  #include <LiteralsTableView.hpp>
8  #include <Logger.hpp>
9  #include <Parser.hpp>
10 #include <SymbolStore.hpp>
11 #include <SyntaxError.hpp>
12 #include <SyntaxTreeView.hpp>
13 #include <Tokenizer.hpp>
14 #include <TokensTableView.hpp>
15 #include <format>
16 #include <iostream>
17
18 int main(int argc, char* argv[]) {
19     SetConsoleOutputCP(CP_UTF8);
20
21     Args args(argc, argv);
22     args.expectString("source", "s", true)
23         .expectFlag("tokens", "t")
24         .expectFlag("tree", "T")
25         .expectFlag("identifiers", "i")
26         .expectFlag("literals", "l")
27         .expectString("dot", "d", false)
28         .parse();
29
30     auto errorFormatter = [](const auto& err) {
31         return std::format(
32             "Error [{}:{}]: {}", err.row + 1, err.column + 1, err.message
33         );
34     };
35
36     SymbolStore symbols;
37     CharacterAttributes attributes;
38
39     FileSource source(args.getString("source"));
40     Logger<Error> tokenizerLogger("Tokenizer", errorFormatter);
41     Tokenizer tokenizer(source, symbols, attributes, tokenizerLogger);
42     tokenizer.scan();
43
44     Logger<SyntaxError> parserLogger("Parser", errorFormatter);
45     Parser parser(symbols, tokenizer.tokens(), parserLogger);
46     parser.parse();
47
48     if (args.getFlag("tokens")) {
49         TokensTableView("\nTokens", symbols, tokenizer.tokens()).print();
50     }
51
52     if (args.getFlag("identifiers")) {
53         IdentifiersTableView("\nIdentifiers", symbols.identifiers()).print();
54     }
55
56     if (args.getFlag("literals")) {
57         LiteralsTableView("\nLiterals", symbols.literals()).print();
58     }
59
60     if (args.getFlag("tree")) {
```

```
61     SyntaxTreeView("\nSyntax Tree", parser.tree()).print();
62 }
63
64 if (!args.getString("dot").empty()) {
65     DotTreeView(parser.tree()).write(args.getString("dot"));
66 }
67
68 return 0;
69 }
```

```
1 #pragma once
2
3 #include <Logger.hpp>
4 #include <ParsingScope.hpp>
5 #include <Queue.hpp>
6 #include <Stack.hpp>
7 #include <SymbolStore.hpp>
8 #include <SyntaxData.hpp>
9 #include <SyntaxError.hpp>
10 #include <Tokenizer.hpp>
11 #include <Tree.hpp>
12
13 class Parser {
14 private:
15     const SymbolStore& _symbols;
16     Logger<SyntaxError>& _logger;
17
18     Queue<Token> _tokens;
19
20     Tree<SyntaxData> _tree;
21     Stack<TreeNode<SyntaxData>*> _nodes;
22
23     // 1. <signal-program> --> <program>
24     void parseSignalProgram();
25     // 2. <program> --> PROGRAM <procedure-identifier>; <block>.
26     void parseProgram();
27     // 3. <block> --> <declarations> BEGIN <statements-list> END
28     void parseBlock();
29     // 4. <statements-list> --> <empty>
30     void parseStatementsList();
31     // 5. <declarations> --> <constant-declarations>
32     void parseDeclarations();
33     // 6. <constant-declarations> --> CONST <constant-declarations-list> |
34     // <empty>
35     void parseConstantDeclarations();
36     // 7. <constant-declarations-list> --> <constant-declaration>
37     // <constant-declarations-list> | <empty>
38     void parseConstantDeclarationsList();
39     // 8. <constant-declaration> --> <constant-identifier> = <constant>;
40     void parseConstantDeclaration();
41     // 9. <constant> --> '<complex-number>'
42     void parseConstant();
43     // 10. <complex-number> --> <left-part> <right-part>
44     void parseComplexNumber();
45     // 11. <left-part> --> <unsigned-integer> | <empty>
46     void parseLeftPart();
47     // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
48     // <empty>
49     void parseRightPart();
50     // 13. <constant-identifier> --> <identifier>
51     void parseConstantIdentifier();
52     // 14. <procedure-identifier> --> <identifier>
53     void parseProcedureIdentifier();
54     // 15. <identifier> --> <letter><string>
55     void parseIdentifier(const std::string& errorMessage);
56     // 17. <unsigned-integer> --> <digit><digits-string>
57     void parseUnsignedInteger();
58
59     void skipToProcedureBody();
60     void skipToBlockEnd();
61     void skipToNextDeclaration();
62
```

```

63     Token expect(const std::string& expected, const std::string& errorMessage);
64     Token expect(SymbolType expected, const std::string& errorMessage);
65
66     bool consider(const std::string& expected);
67     bool consider(SymbolType expected);
68
69     [[noreturn]] void fail(const std::string& message);
70     [[noreturn]] void fail(const std::string& message, const Token& token);
71
72     [[nodiscard]] ParsingScope grow(const SyntaxData& data);
73
74 public:
75     Parser(
76         const SymbolStore& symbols,
77         const std::vector<Token>& tokens,
78         Logger<SyntaxError>& logger
79     );
80
81     void parse();
82     const Tree<SyntaxData>& tree() const;
83 };

```

```
1 #include "Parser.hpp"
2
3 #include <format>
4
5 Parser::Parser(
6     const SymbolStore& symbols,
7     const std::vector<Token>& tokens,
8     Logger<SyntaxError>& logger
9 ) :
10     _symbols(symbols),
11     _logger(logger),
12     _tokens(tokens),
13     _tree(Tree<SyntaxData>({.symbol = "<signal-program>", .rule = "S"})) {
14     _nodes.push(&_tree.root());
15 }
16
17 void Parser::parse() {
18     parseSignalProgram();
19 }
20
21 // 1. <signal-program> --> <program>
22 void Parser::parseSignalProgram() {
23     auto scope = grow({.symbol = "<program>", .rule = "1"});
24
25     try {
26         parseProgram();
27     } catch (const SyntaxError&) {
28         // If parsing fails at the top level, we can't really recover,
29         // so we just clear the remaining tokens to prevent cascading errors.
30         _tokens.clear();
31     }
32
33     if (!_tokens.isEmpty()) {
34         auto token = _tokens.peek();
35
36         _logger.message(SyntaxError(
37             SyntaxError::UnexpectedSymbolsAfterEndOfProgram,
38             token->row,
39             token->column
40         ));
41     }
42 }
43
44 // 2. <program> --> PROGRAM <procedure-identifier>; <block>.
45 void Parser::parseProgram() {
46     auto scope = grow({
47         .symbol = "PROGRAM <procedure-identifier>; <block>.",
48         .rule = "2",
49     });
50
51     try {
52         expect(Keyword::Program, SyntaxError::MustStartWithProgram);
53         parseProcedureIdentifier();
54         expect(Delimiter::Semicolon, SyntaxError::ExpectedSemicolon);
55     } catch (const SyntaxError&) {
56         skipToProcedureBody();
57     }
58
59     try {
60         parseBlock();
61     } catch (const SyntaxError&) {
62         skipToBlockEnd();
63     }
```

```

63     }
64
65     expect(Delimiter::Dot, SyntaxError::ExpectedDot);
66 }
67
68 // 3. <block> --> <declarations> BEGIN <statements-list> END
69 void Parser::parseBlock() {
70     auto scope = grow({
71         .symbol = "<declarations> BEGIN <statements-list> END",
72         .rule = "3",
73     });
74
75     try {
76         parseDeclarations();
77     } catch (const SyntaxError& error) {
78         if (error.message == SyntaxError::ExpectedConstKeyword) {
79             skipToProcedureBody();
80         } else {
81             throw error;
82         }
83     }
84
85     expect(Keyword::Begin, SyntaxError::ExpectedBeginKeyword);
86     parseStatementsList();
87     expect(Keyword::End, SyntaxError::ExpectedEndKeyword);
88 }
89
90 // 4. <statements-list> --> <empty>
91 void Parser::parseStatementsList() {
92     auto scope = grow({
93         .symbol = "<empty>",
94         .rule = "4",
95     });
96 }
97
98 // 5. <declarations> --> <constant-declarations>
99 void Parser::parseDeclarations() {
100     auto scope = grow({
101         .symbol = "<constant-declarations>",
102         .rule = "5",
103     });
104
105     parseConstantDeclarations();
106 }
107
108 // 6. <constant-declarations> --> CONST <constant-declarations-list> | <empty>
109 void Parser::parseConstantDeclarations() {
110     if (consider(Keyword::Const)) {
111         auto scope = grow({
112             .symbol = "CONST <constant-declarations-list>",
113             .rule = "6",
114         });
115
116         expect(Keyword::Const, SyntaxError::ExpectedConstKeyword);
117         parseConstantDeclarationsList();
118     }
119
120     auto scope = grow({
121         .symbol = "<empty>",
122         .rule = "6",
123     });
124 }
125
126 // 7. <constant-declarations-list> --> <constant-declaration>
127 // <constant-declarations-list> | <empty>
128 void Parser::parseConstantDeclarationsList() {
129     if (!consider(Keyword::Begin)) {

```

```

130     auto scope = grow({
131         .symbol = "<constant-declaration> <constant-declarations-list>",
132         .rule = "7",
133     });
134
135     try {
136         parseConstantDeclaration();
137     } catch (const SyntaxError&) {
138         skipToNextDeclaration();
139     }
140
141     parseConstantDeclarationsList();
142 } else {
143     auto scope = grow({.symbol = "<empty>", .rule = "7"});
144 }
145 }
146
147 // 8. <constant-declaration> --> <constant-identifier> = <constant>;
148 void Parser::parseConstantDeclaration() {
149     auto scope = grow({
150         .symbol = "<constant-identifier> = <constant>;",
151         .rule = "8",
152     });
153
154     parseConstantIdentifier();
155     expect(Delimiter::Equals, SyntaxError::ExpectedEquals);
156     parseConstant();
157     try {
158         expect(Delimiter::Semicolon, SyntaxError::ExpectedConstantSemicolon);
159     } catch (const SyntaxError&) {
160         // Implicitly add a semicolon to recover and continue parsing the next declarations.
161         // TODO: check that this does not backfire for severely mangled declarations.
162     }
163 }
164
165 // 9. <constant> --> '<complex-number>'
166 void Parser::parseConstant() {
167     auto scope = grow({
168         .symbol = "'<complex-number>' ",
169         .rule = "9",
170     });
171
172     expect(Delimiter::Quote, SyntaxError::ExpectedOpeningQuote);
173     parseComplexNumber();
174     expect(Delimiter::Quote, SyntaxError::ExpectedClosingQuote);
175 }
176
177 // 10. <complex-number> --> <left-part> <right-part>
178 void Parser::parseComplexNumber() {
179     auto scope = grow({
180         .symbol = "<left-part> <right-part>",
181         .rule = "10",
182     });
183
184     parseLeftPart();
185     parseRightPart();
186 }
187
188 // 11. <left-part> --> <unsigned-integer> | <empty>
189 void Parser::parseLeftPart() {
190     if (consider(SymbolType::Literal)) {
191         auto scope = grow({
192             .symbol = "<unsigned-integer>",
193             .rule = "11",
194         });
195
196         parseUnsignedInteger();

```

```

197     } else {
198         auto scope = grow({.symbol = "<empty>", .rule = "11"});
199     }
200 }
201
202 // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
203 // <empty>
204 void Parser::parseRightPart() {
205     if (consider(Delimiter::Comma)) {
206         auto scope = grow({
207             .symbol = ",<unsigned-integer>",
208             .rule = "12",
209         });
210
211         expect(Delimiter::Comma, SyntaxError::ExpectedComma);
212         parseUnsignedInteger();
213         return;
214     }
215
216     if (consider(MultiDelimiter::Exp)) {
217         auto scope = grow({
218             .symbol = "$EXP( <unsigned-integer> )",
219             .rule = "12",
220         });
221
222         expect(MultiDelimiter::Exp, SyntaxError::ExpectedExp);
223         expect(Delimiter::OpenParenthesis, SyntaxError::ExpectedOpenParen);
224         parseUnsignedInteger();
225         expect(Delimiter::CloseParenthesis, SyntaxError::ExpectedCloseParen);
226         return;
227     }
228
229     auto scope = grow({.symbol = "<empty>", .rule = "12"});
230 }
231
232 // 13. <constant-identifier> --> <identifier>
233 void Parser::parseConstantIdentifier() {
234     auto scope = grow({
235         .symbol = "<identifier>",
236         .rule = "13"
237     });
238
239     parseIdentifier(SyntaxError::ExpectedConstantIdentifier);
240 }
241
242 // 14. <procedure-identifier> --> <identifier>
243 void Parser::parseProcedureIdentifier() {
244     auto scope = grow({.symbol = "<identifier>", .rule = "14"});
245
246     parseIdentifier(SyntaxError::ExpectedProcedureIdentifier);
247 }
248
249 // 15. <identifier> --> <letter><string>
250 void Parser::parseIdentifier(const std::string& errorMessage) {
251     auto token = expect(SymbolType::Identifier, errorMessage);
252
253     auto scope = grow({
254         .symbol = _symbols.lookup(token.code),
255         .rule = "15",
256     });
257 }
258
259 // 17. <unsigned-integer> --> <digit><digits-string>
260 void Parser::parseUnsignedInteger() {
261     auto token = expect(SymbolType::Literal, SyntaxError::ExpectedUnsignedInteger);
262
263     auto scope = grow({

```

```

264         .symbol = _symbols.lookup(token.code),
265         .rule = "17",
266     });
267 }
268
269 // --- Error recovery ---
270
271 void Parser::skipToProcedureBody() {
272     auto found = _tokens.popUntil([&](const Token& token) {
273         const auto& symbol = _symbols.lookup(token.code);
274         return symbol == Keyword::Const || symbol == Keyword::Begin;
275     });
276
277     if (!found) {
278         fail(SyntaxError::UnexpectedEndOfFile);
279     }
280 }
281
282 void Parser::skipToBlockEnd() {
283     auto found = _tokens.popUntil([&](const Token& token) {
284         const auto& symbol = _symbols.lookup(token.code);
285         return symbol == Keyword::End || symbol == Delimiter::Dot;
286     });
287
288     if (!found) {
289         fail(SyntaxError::UnexpectedEndOfFile);
290     }
291
292     if (_symbols.lookup(found->code) == Keyword::End) {
293         _tokens.pop();
294     }
295 }
296
297 void Parser::skipToNextDeclaration() {
298     auto found = _tokens.popUntil([&](const Token& token) {
299         const auto& symbol = _symbols.lookup(token.code);
300         return symbol == Delimiter::Semicolon || symbol == Keyword::Begin;
301     });
302
303     if (!found) {
304         fail(SyntaxError::UnexpectedEndOfFile);
305     }
306
307     if (_symbols.lookup(found->code) == Delimiter::Semicolon) {
308         _tokens.pop();
309     }
310 }
311
312 // --- Utilities ---
313
314 Token Parser::expect(
315     const std::string& expected,
316     const std::string& errorMessage
317 ) {
318     auto token = _tokens.peek();
319
320     if (!token) {
321         fail(errorMessage);
322     }
323
324     if (_symbols.lookup(token->code) != expected) {
325         fail(errorMessage, *token);
326     }
327
328     return *_tokens.pop();
329 }
330

```

```

331 Token Parser::expect(SymbolType expected, const std::string& errorMessage) {
332     auto token = _tokens.peek();
333
334     if (!token) {
335         fail(errorMessage);
336     }
337
338     if (_symbols.lookupType(token->code) != expected) {
339         fail(errorMessage, *token);
340     }
341
342     return *_tokens.pop();
343 }
344
345 bool Parser::consider(const std::string& expected) {
346     auto token = _tokens.peek();
347
348     return token && _symbols.lookup(token->code) == expected;
349 }
350
351 bool Parser::consider(SymbolType expected) {
352     auto token = _tokens.peek();
353
354     return token && _symbols.lookupType(token->code) == expected;
355 }
356
357 void Parser::fail(const std::string& message) {
358     SyntaxError error(message, 0, 0);
359     _logger.message(error);
360     throw error;
361 }
362
363 void Parser::fail(const std::string& message, const Token& token) {
364     SyntaxError error(message, token.row, token.column);
365     _logger.message(error);
366     throw error;
367 }
368
369 ParsingScope Parser::grow(const SyntaxData& data) {
370     auto parentNode = _nodes.peek();
371     auto& node = (*parentNode)->grow(data);
372     _nodes.push(&node);
373     return ParsingScope(_nodes);
374 }
375
376 const Tree<SyntaxData>& Parser::tree() const {
377     return _tree;
378 }

```

src\parser\data\ParsingScope\ParsingScope.hpp

```
1 #pragma once
2
3 #include <Stack.hpp>
4 #include <SyntaxData.hpp>
5 #include <Tree.hpp>
6
7 class ParsingScope {
8     Stack<TreeNode<SyntaxData>*>* _stack;
9
10 public:
11     explicit ParsingScope(Stack<TreeNode<SyntaxData>*>& stack);
12     ~ParsingScope();
13
14     ParsingScope(ParsingScope&& other) noexcept;
15     ParsingScope(const ParsingScope&) = delete;
16     ParsingScope& operator=(const ParsingScope&) = delete;
17     ParsingScope& operator=(ParsingScope&&) = delete;
18 };
```

src\parser\data\ParsingScope\ParsingScope.cpp

```
1 #include "ParsingScope.hpp"
2
3 ParsingScope::ParsingScope(Stack<TreeNode<SyntaxData>*>& stack) :
4     _stack(&stack) {}
5
6 ParsingScope::~ParsingScope() {
7     if (_stack) {
8         _stack->pop();
9     }
10 }
11
12 ParsingScope::ParsingScope(ParsingScope&& other) noexcept :
13     _stack(other._stack) {
14     other._stack = nullptr;
15 }
```

src\parser\data\SyntaxData\SyntaxData.hpp

```
1 #pragma once
2
3 #include <string>
4
5 struct SyntaxData {
6     std::string symbol;
7     std::string rule;
8 };
```

```
1 #pragma once
2
3 #include <cstdint>
4 #include <stdexcept>
5 #include <string>
6
7 struct SyntaxError : std::runtime_error {
8     std::string message;
9     size_t row;
10    size_t column;
11
12    SyntaxError(std::string message, size_t row, size_t column);
13
14    // Error message constants
15    static constexpr auto MustStartWithProgram =
16        "Program must start with 'PROGRAM' keyword";
17    static constexpr auto ExpectedProcedureIdentifier =
18        "Expected procedure identifier after 'PROGRAM' keyword";
19    static constexpr auto ExpectedSemicolon =
20        "Expected ';' after procedure identifier";
21    static constexpr auto ExpectedDot =
22        "Expected '.' at the end of the program";
23    static constexpr auto ExpectedBeginKeyword =
24        "Expected 'BEGIN' keyword before statements list";
25    static constexpr auto ExpectedEndKeyword =
26        "Expected 'END' keyword at the end of the block";
27    static constexpr auto ExpectedConstKeyword =
28        "Expected 'CONST' keyword before constant declarations";
29    static constexpr auto ExpectedIdentifier = "Expected identifier";
30    static constexpr auto ExpectedConstantIdentifier =
31        "Expected constant identifier in constant declaration";
32    static constexpr auto ExpectedEquals =
33        "Expected '=' in constant declaration";
34    static constexpr auto ExpectedConstantSemicolon =
35        "Expected ';' at the end of constant declaration";
36    static constexpr auto ExpectedOpeningQuote =
37        "Expected '"' (single quote) at the beginning of constant declaration";
38    static constexpr auto ExpectedClosingQuote =
39        "Expected '"' (single quote) at the end of constant declaration";
40    static constexpr auto ExpectedUnsignedInteger =
41        "Expected unsigned integer literal";
42    static constexpr auto ExpectedComma = "Expected ',' in complex number";
43    static constexpr auto ExpectedExp = "Expected '$EXP' in complex number";
44    static constexpr auto ExpectedOpenParen =
45        "Expected '(' after '$EXP' in complex number";
46    static constexpr auto ExpectedCloseParen =
47        "Expected ')' after exponent in complex number";
48    static constexpr auto UnexpectedEndOfFile = "Unexpected end of file";
49    static constexpr auto UnexpectedSymbolsAfterEndOfProgram =
50        "Unexpected symbols after end of program";
51 };
```

src\parser\data\SyntaxError\SyntaxError.cpp

```
1 #include "SyntaxError.hpp"
2
3 #include <utility>
4
5 SyntaxError::SyntaxError(std::string message, size_t row, size_t column) :
6     std::runtime_error(message),
7     message(std::move(message)),
8     row(row),
9     column(column) {}
```

src\parser\view\DotTreeView\DotTreeView.hpp

```
1 #pragma once
2
3 #include <string>
4
5 #include <Parser.hpp>
6
7 class DotTreeView {
8 private:
9     const Tree<SyntaxData>& _tree;
10
11     void writeNode(
12         std::ostream& out,
13         const TreeNode<SyntaxData>& node,
14         int& id
15     ) const;
16
17 public:
18     DotTreeView(const Tree<SyntaxData>& tree);
19
20     void write(const std::string& filename) const;
21 };
```

src\parser\view\DotTreeView\DotTreeView.cpp

```
1 #include "DotTreeView.hpp"
2
3 #include <fstream>
4
5 DotTreeView::DotTreeView(const Tree<SyntaxData>& tree) : _tree(tree) {}
6
7 void DotTreeView::writeNode(
8     std::ostream& out,
9     const TreeNode<SyntaxData>& node,
10    int& id
11 ) const {
12     int nodeId = id++;
13     const auto& data = node.data();
14
15     out << "    n" << nodeId
16         << " [label=\"" << data.symbol << "\"]\n";
17
18     for (const auto& child : node.children()) {
19         int childId = id;
20         writeNode(out, *child, id);
21
22         int ruleId = id++;
23         out << "    r" << ruleId
24             << " [label=\"" << child->data().rule
25                 << "\" shape=circle width=0.3 fixedsize=true"
26                 << " fontsize=10]\n";
27
28         out << "    n" << nodeId << " -> r" << ruleId
29             << " [arrowhead=none]\n";
30         out << "    r" << ruleId << " -> n" << childId << "\n";
31     }
32 }
33
34 void DotTreeView::write(const std::string& filename) const {
35     std::ofstream out(filename);
36
37     out << "digraph SyntaxTree {\n"
38         << "    rankdir=TB\n"
39         << "    node [shape=none fontname=\"Consolas\" fontsize=11]\n"
40         << "    edge [arrowsize=0.6]\n\n";
41
42     int id = 0;
43     writeNode(out, _tree.root(), id);
44
45     out << "}\n";
46 }
```

src\parser\view\SyntaxTreeView\SyntaxTreeView.hpp

```
1 #pragma once
2
3 #include <string>
4
5 #include <Parser.hpp>
6 #include <TreeView.hpp>
7
8 class SyntaxTreeView {
9 private:
10     TreeView<SyntaxData> _tree;
11
12 public:
13     SyntaxTreeView(std::string title, const Tree<SyntaxData>& tree);
14
15     void print() const;
16 };
```

```
1 #include "SyntaxTreeView.hpp"
2
3 #include <format>
4
5 SyntaxTreeView::SyntaxTreeView(
6     std::string title,
7     const Tree<SyntaxData>& tree
8 ) :
9     _tree(std::move(title), tree) {
10     _tree
11         .setNodeFormatter([](const auto& data) { return data.symbol; })
12         .setEdgeFormatter([](const auto& data) {
13             return std::format("({})", data.rule);
14         });
15 }
16
17 void SyntaxTreeView::print() const {
18     _tree.print();
19 }
```

src\symbols\symbols.constants.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 constexpr size_t ASCII_TABLE_SIZE = 256;
6
7 namespace Keyword {
8     constexpr auto Program = "PROGRAM";
9     constexpr auto Const = "CONST";
10    constexpr auto Begin = "BEGIN";
11    constexpr auto End = "END";
12 } // namespace Keyword
13
14 namespace MultiDelimiter {
15     constexpr auto Exp = "$EXP";
16 }
17
18 namespace Delimiter {
19     constexpr auto Semicolon = ";";
20     constexpr auto Dot = ".";
21     constexpr auto Equals = "=";
22     constexpr auto Quote = "'";
23     constexpr auto Comma = ",";
24     constexpr auto OpenParenthesis = "(";
25     constexpr auto CloseParenthesis = ")";
26 } // namespace Delimiter
```

```
1  #pragma once
2
3  #include <array>
4  #include <string>
5  #include <unordered_map>
6  #include <vector>
7
8  #include "symbols.constants.hpp"
9
10 enum class SymbolType {
11     Ascii,
12     MultiDelimiter,
13     Keyword,
14     Literal,
15     Identifier,
16 };
17
18 const size_t DELIMITERS_OFFSET = 0;
19 const size_t MULTI_DELIMITERS_OFFSET = ASCII_TABLE_SIZE;
20 const size_t KEYWORDS_OFFSET = 300;
21 const size_t LITERALS_OFFSET = 500;
22 const size_t IDENTIFIERS_OFFSET = 1000;
23 const size_t MAX_TOKENS = 2000;
24
25 class SymbolStore {
26 private:
27     std::array<std::string, MAX_TOKENS> symbols;
28     std::unordered_map<std::string, size_t> _multiDelimiters;
29     std::unordered_map<std::string, size_t> _keywords;
30     std::unordered_map<std::string, size_t> _identifiers;
31     std::unordered_map<std::string, size_t> _literals;
32
33     void declareAscii(char character);
34     void declareMultiDelimiter(const std::string& delimiter);
35     void declareKeyword(const std::string& keyword);
36
37 public:
38     SymbolStore();
39
40     size_t resolveMultiDelimiter(const std::string& delimiter);
41     size_t resolveKeyword(const std::string& keyword);
42     size_t resolveIdentifier(const std::string& identifier);
43     size_t resolveLiteral(const std::string& literal);
44
45     bool isKeyword(const std::string& token) const;
46     bool isMultiDelimiter(const std::string& token) const;
47
48     const std::string& lookup(size_t code) const;
49     SymbolType lookupType(size_t code) const;
50     std::vector<std::pair<std::string, size_t>> identifiers() const;
51     std::vector<std::pair<std::string, size_t>> literals() const;
52 };
```

```
1 #include "SymbolStore.hpp"
2
3 #include <algorithm>
4 #include <format>
5 #include <stdexcept>
6
7 SymbolStore::SymbolStore() {
8     // Initialize ASCII symbols
9     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
10         declareAscii(static_cast<char>(i));
11     }
12
13     // Initialize multi-character delimiters
14     declareMultiDelimiter(MultiDelimiter::Exp);
15
16     // Initialize keywords
17     declareKeyword(Keyword::Program);
18     declareKeyword(Keyword::Const);
19     declareKeyword(Keyword::Begin);
20     declareKeyword(Keyword::End);
21 }
22
23 void SymbolStore::declareAscii(const char character) {
24     symbols[static_cast<unsigned char>(character)] = std::string(1, character);
25 }
26
27 void SymbolStore::declareMultiDelimiter(const std::string& delimiter) {
28     if (_multiDelimiters.contains(delimiter)) {
29         throw std::invalid_argument(
30             std::format("Multi-delimiter '{}{}' is already declared", delimiter)
31         );
32     }
33
34     size_t code = MULTI_DELIMITERS_OFFSET + _multiDelimiters.size();
35
36     if (code >= KEYWORDS_OFFSET) {
37         throw std::overflow_error(
38             "Exceeded maximum number of multi-delimiters"
39         );
40     }
41
42     _multiDelimiters[delimiter] = code;
43     symbols[code] = delimiter;
44 }
45
46 size_t SymbolStore::resolveMultiDelimiter(const std::string& delimiter) {
47     if (!_multiDelimiters.contains(delimiter)) {
48         throw std::invalid_argument(
49             std::format("'{}{}' is not a multi-delimiter", delimiter)
50         );
51     }
52
53     return _multiDelimiters[delimiter];
54 }
55
56 void SymbolStore::declareKeyword(const std::string& keyword) {
57     if (_keywords.contains(keyword)) {
58         throw std::invalid_argument(
59             std::format("Keyword '{}{}' is already declared", keyword)
60         );
61     }
62 }
```

```

63     size_t code = KEYWORDS_OFFSET + _keywords.size();
64
65     if (code >= LITERALS_OFFSET) {
66         throw std::overflow_error("Exceeded maximum number of keywords");
67     }
68
69     _keywords[keyword] = code;
70     symbols[code] = keyword;
71 }
72
73 size_t SymbolStore::resolveKeyword(const std::string& keyword) {
74     if (!_keywords.contains(keyword)) {
75         throw std::invalid_argument(
76             std::format("{} is not a keyword", keyword)
77         );
78     }
79
80     return _keywords[keyword];
81 }
82
83 size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
84     if (!_identifiers.contains(identifier)) {
85         size_t code = IDENTIFIERS_OFFSET + _identifiers.size();
86
87         if (code >= MAX_TOKENS) {
88             throw std::overflow_error("Exceeded maximum number of identifiers");
89         }
90
91         _identifiers[identifier] = code;
92         symbols[code] = identifier;
93     }
94
95     return _identifiers[identifier];
96 }
97
98 size_t SymbolStore::resolveLiteral(const std::string& literal) {
99     if (!_literals.contains(literal)) {
100         size_t code = LITERALS_OFFSET + _literals.size();
101
102         if (code >= IDENTIFIERS_OFFSET) {
103             throw std::overflow_error("Exceeded maximum number of literals");
104         }
105
106         _literals[literal] = code;
107         symbols[code] = literal;
108     }
109
110     return _literals[literal];
111 }
112
113 bool SymbolStore::isKeyword(const std::string& token) const {
114     return _keywords.contains(token);
115 }
116
117 bool SymbolStore::isMultiDelimiter(const std::string& token) const {
118     return _multiDelimiters.contains(token);
119 }
120
121 SymbolType SymbolStore::lookupType(size_t code) const {
122     if (code < MULTI_DELIMITERS_OFFSET) {
123         return SymbolType::Ascii;
124     }
125
126     if (code < KEYWORDS_OFFSET) {
127         return SymbolType::MultiDelimiter;
128     }
129

```

```

130     if (code < LITERALS_OFFSET) {
131         return SymbolType::Keyword;
132     }
133
134     if (code < IDENTIFIERS_OFFSET) {
135         return SymbolType::Literal;
136     }
137
138     if (code < MAX_TOKENS) {
139         return SymbolType::Identifier;
140     }
141
142     throw std::out_of_range(
143         std::format("Symbol code {} is out of range", code)
144     );
145 }
146
147 const std::string& SymbolStore::lookup(size_t code) const {
148     if (code >= symbols.size()) {
149         throw std::out_of_range(
150             std::format("Symbol code {} is out of range", code)
151         );
152     }
153
154     return symbols[code];
155 }
156
157 std::vector<std::pair<std::string, size_t>> SymbolStore::identifiers() const {
158     std::vector<std::pair<std::string, size_t>> result;
159
160     for (const auto& pair : _identifiers) {
161         result.emplace_back(pair.first, pair.second);
162     }
163
164     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
165         return a.second < b.second;
166     });
167
168     return result;
169 }
170
171 std::vector<std::pair<std::string, size_t>> SymbolStore::literals() const {
172     std::vector<std::pair<std::string, size_t>> result;
173
174     for (const auto& pair : _literals) {
175         result.emplace_back(pair.first, pair.second);
176     }
177
178     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
179         return a.second < b.second;
180     });
181
182     return result;
183 }

```

```
1 #pragma once
2
3 #include <array>
4
5 #include "symbols.constants.hpp"
6
7 enum class Attribute {
8     Whitespace,
9     Digit,
10    Letter,
11    Delimiter,
12    MultiDelimiter,
13    Comment,
14    Invalid,
15 };
16
17 class CharacterAttributes {
18 private:
19     std::array<Attribute, ASCII_TABLE_SIZE> attributes;
20
21 public:
22     CharacterAttributes();
23
24     [[nodiscard]] Attribute lookup(char character) const;
25 };
```

```
1 #include "CharacterAttributes.hpp"
2
3 CharacterAttributes::CharacterAttributes() {
4     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
5         attributes[i] = Attribute::Invalid;
6     }
7
8     // Whitespace
9     attributes[' '] = Attribute::Whitespace;
10    attributes['\t'] = Attribute::Whitespace;
11    attributes['\n'] = Attribute::Whitespace;
12    attributes['\v'] = Attribute::Whitespace;
13    attributes['\f'] = Attribute::Whitespace;
14    attributes['\r'] = Attribute::Whitespace;
15
16    // Digits
17    for (char ch = '0'; ch <= '9'; ch++) {
18        attributes[ch] = Attribute::Digit;
19    }
20
21    // Letters (A-Z)
22    for (char ch = 'A'; ch <= 'Z'; ch++) {
23        attributes[ch] = Attribute::Letter;
24    }
25
26    // Delimiters
27    attributes[';'] = Attribute::Delimiter;
28    attributes['.'] = Attribute::Delimiter;
29    attributes['='] = Attribute::Delimiter;
30    attributes['\'] = Attribute::Delimiter;
31    attributes[','] = Attribute::Delimiter;
32    attributes['\'] = Attribute::Delimiter;
33
34    // Multi-delimiters
35    attributes['$'] = Attribute::MultiDelimiter;
36
37    // Comment start
38    attributes['('] = Attribute::Comment;
39 }
40
41 Attribute CharacterAttributes::lookup(char character) const {
42     return attributes[static_cast<unsigned char>(character)];
43 }
```

```
1 #pragma once
2
3 #include <cstdint>
4
5 class Cursor {
6 private:
7     uint8_t tabSize = 4;
8
9     unsigned int _row = 0;
10    unsigned int _column = 0;
11
12 public:
13     Cursor();
14     Cursor(uint8_t tabSize);
15
16     void advance(char character);
17     [[nodiscard]] unsigned int row() const;
18     [[nodiscard]] unsigned int column() const;
19 };
```

```
1 #include "Cursor.hpp"
2
3 Cursor::Cursor() : Cursor(4) {}
4
5 Cursor::Cursor(uint8_t tabSize) : tabSize(tabSize) {};
6
7 void Cursor::advance(char character) {
8     switch (character) {
9         case '\r':
10             break;
11
12         case '\n':
13             _row++;
14             _column = 0;
15             break;
16
17         case '\t':
18             _column += tabSize - _column % tabSize;
19             break;
20
21         default:
22             _column++;
23             break;
24     }
25 }
26
27 unsigned int Cursor::row() const {
28     return _row;
29 }
30
31 unsigned int Cursor::column() const {
32     return _column;
33 }
```

src\tokenizer\data\FileSource\FileSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <fstream>
5 #include "Source.hpp"
6
7 class FileSource : public Source {
8 private:
9     std::ifstream file;
10     char character;
11
12 public:
13     FileSource(const std::string& filePath);
14
15     char current() override;
16     char read() override;
17     bool done() override;
18 };
```

src\tokenizer\data\FileSource\FileSource.cpp

```
1 #include "FileSource.hpp"
2
3 #include <stdexcept>
4
5 FileSource::FileSource(const std::string& filePath) :
6     file(filePath),
7     character(static_cast<char>(file.get())) {
8     if (!file.is_open()) {
9         throw std::runtime_error("Could not open file: " + filePath);
10    }
11 }
12
13 char FileSource::current() {
14     return character;
15 }
16
17 char FileSource::read() {
18     _cursor.advance(character);
19     character = static_cast<char>(file.get());
20
21     return character;
22 }
23
24 bool FileSource::done() {
25     return file.eof();
26 }
```

src\tokenizer\data\Source\Source.hpp

```
1 #pragma once
2
3 #include <Cursor.hpp>
4
5 class Source {
6 protected:
7     Cursor _cursor;
8
9 public:
10     Source();
11     Source(Cursor cursor);
12
13     virtual ~Source() = default;
14
15     virtual char current() = 0;
16     virtual char read() = 0;
17     virtual bool done() = 0;
18
19     [[nodiscard]] const Cursor& cursor() const;
20 };
```

src\tokenizer\data\Source\Source.cpp

```
1 #include "Source.hpp"
2
3 Source::Source() = default;
4
5 Source::Source(Cursor cursor) : _cursor(cursor) {}
6
7 const Cursor& Source::cursor() const {
8     return _cursor;
9 }
```

src\tokenizer\data\StringSource\StringSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include "Source.hpp"
5
6 class StringSource : public Source {
7 private:
8     std::string source;
9     std::string::iterator iterator;
10
11 public:
12     StringSource(std::string source);
13
14     char current() override;
15     char read() override;
16     bool done() override;
17 };
```

```
1 #include "StringSource.hpp"
2
3 StringSource::StringSource(std::string source) :
4     source(std::move(source)),
5     iterator(this->source.begin()) {}
6
7 char StringSource::current() {
8     return *iterator;
9 }
10
11 char StringSource::read() {
12     _cursor.advance(*iterator);
13     iterator++;
14
15     return *iterator;
16 }
17
18 bool StringSource::done() {
19     return iterator == source.end();
20 }
```

src\tokenizer\data\Tokenizer\Tokenizer.hpp

```
1  #pragma once
2
3  #include <unordered_map>
4  #include <vector>
5
6  #include "CharacterAttributes.hpp"
7  #include "Logger.hpp"
8  #include "Source.hpp"
9  #include "SymbolStore.hpp"
10
11 struct Token {
12     size_t code;
13     size_t row;
14     size_t column;
15 };
16
17 struct Error {
18     std::string message;
19     size_t row;
20     size_t column;
21 };
22
23 class Tokenizer {
24 private:
25     Source& source;
26     SymbolStore& symbols;
27     CharacterAttributes& attributes;
28     Logger<Error>& _logger;
29
30     char character;
31     std::string token;
32     size_t code;
33
34     std::vector<Token> _tokens;
35
36     void scanWhitespaces();
37     void scanInteger();
38     void scanString();
39     void scanComment();
40     void scanMultiDelimiter();
41     void scanDelimiter();
42     void scanInvalid();
43
44 public:
45     Tokenizer(
46         Source& source,
47         SymbolStore& symbols,
48         CharacterAttributes& attributes,
49         Logger<Error>& logger
50     );
51
52     void scan();
53
54     void addToken(const Token& token);
55     void addError(const Error& error);
56
57     [[nodiscard]] const std::vector<Token>& tokens() const;
58 };
```

src\tokenizer\data\Tokenizer\Tokenizer.cpp

```
1 #include "Tokenizer.hpp"
2
3 #include <format>
4
5 #include "CharacterAttributes.hpp"
6
7 // Automata state: START
8 Tokenizer::Tokenizer(
9     Source& source,
10     SymbolStore& symbols,
11     CharacterAttributes& attributes,
12     Logger<Error>& logger
13 ) :
14     source(source),
15     symbols(symbols),
16     attributes(attributes),
17     _logger(logger),
18     character(source.current()) {}
19
20 void Tokenizer::scan() {
21     // Automata state: LOOP
22     while (!source.done()) {
23         switch (attributes.lookup(character)) {
24             // Automata state: WHITESPACE
25             case Attribute::Whitespace:
26                 scanWhitespaces();
27                 break;
28
29             // Automata state: INTEGER
30             case Attribute::Digit:
31                 scanInteger();
32                 break;
33
34             // Automata state: STRING
35             case Attribute::Letter:
36                 scanString();
37                 break;
38
39             // Automata state: MULTI_DELIMITER
40             case Attribute::MultiDelimiter:
41                 scanMultiDelimiter();
42                 break;
43
44             // Automata state: COMMENT_START
45             case Attribute::Comment:
46                 scanComment();
47                 break;
48
49             // Automata state: DELIMITER
50             case Attribute::Delimiter:
51                 scanDelimiter();
52                 break;
53
54             // Automata state: ERROR
55             case Attribute::Invalid:
56                 scanInvalid();
57                 break;
58         }
59     }
60     // Automata state: END
61 }
62
```

```

63 void Tokenizer::scanWhitespaces() {
64     while (!source.done() &&
65         attributes.lookup(character) == Attribute::Whitespace) {
66         character = source.read();
67     }
68 }
69
70 void Tokenizer::scanInteger() {
71     while (!source.done() && attributes.lookup(character) == Attribute::Digit) {
72         token += character;
73         character = source.read();
74     };
75
76     // Automata state: WRITE
77     code = symbols.resolveLiteral(token);
78
79     addToken({
80         .code = code,
81         .row = source.cursor().row(),
82         .column = source.cursor().column() -
83             token.length(), // Point to first character of the token
84     });
85
86     token.clear();
87 }
88
89 void Tokenizer::scanString() {
90     while (!source.done() &&
91         (attributes.lookup(character) == Attribute::Letter ||
92         attributes.lookup(character) == Attribute::Digit)) {
93         token += character;
94         character = source.read();
95     }
96
97     // Automata state: WRITE
98     if (symbols.isKeyword(token)) {
99         code = symbols.resolveKeyword(token);
100     } else {
101         code = symbols.resolveIdentifier(token);
102     }
103
104     addToken({
105         .code = code,
106         .row = source.cursor().row(),
107         .column = source.cursor().column() -
108             token.length(), // Point to first character of the token
109     });
110
111     token.clear();
112 }
113
114 void Tokenizer::scanMultiDelimiter() {
115     size_t startRow = source.cursor().row();
116     size_t startCol = source.cursor().column();
117
118     token += character;
119     character = source.read();
120
121     if (source.done() || attributes.lookup(character) != Attribute::Letter) {
122         addError({
123             .message = std::format("Invalid use of '$'"),
124             .row = startRow,
125             .column = startCol,
126         });
127
128         token.clear();
129

```

```

130     return;
131 }
132
133 while (!source.done() &&
134        (attributes.lookup(character) == Attribute::Letter)) {
135     token += character;
136     character = source.read();
137 }
138
139 if (!symbols.isMultiDelimiter(token)) {
140     addError({
141         .message = std::format("Invalid use of '$'"),
142         .row = startRow,
143         .column = startCol,
144     });
145
146     token.clear();
147     return;
148 }
149
150 code = symbols.resolveMultiDelimiter(token);
151
152 // Automata state: WRITE
153 addToken({
154     .code = code,
155     .row = startRow,
156     .column = startCol,
157 });
158
159 token.clear();
160 }
161
162 void Tokenizer::scanComment() {
163     size_t startRow = source.cursor().row();
164     size_t startCol = source.cursor().column();
165
166     character = source.read();
167
168     // Automata state: WRITE
169     if (character != '*') {
170         addToken({
171             .code = static_cast<size_t>('('),
172             .row = startRow,
173             .column = startCol,
174         });
175
176         return;
177     }
178
179     character = source.read();
180
181     // Automata state: COMMENT
182     while (!source.done()) {
183         // Automata state: COMMENT_CLOSE
184         if (character == '*') {
185             character = source.read();
186
187             // Automata state: COMMENT_END
188             if (character == ')') {
189                 character = source.read();
190
191                 return;
192             }
193         } else {
194             character = source.read();
195         }
196     }

```

```

197
198     // Automata state: ERROR
199     addError({
200         .message = "Comment not closed",
201         .row = startRow,
202         .column = startCol,
203     });
204 }
205
206 void Tokenizer::scanDelimiter() {
207     addToken({
208         .code = static_cast<size_t>(character),
209         .row = source.cursor().row(),
210         .column = source.cursor().column(),
211     });
212
213     character = source.read();
214 }
215
216 void Tokenizer::scanInvalid() {
217     addError({
218         .message = std::format("Invalid character '{}'", character),
219         .row = source.cursor().row(),
220         .column = source.cursor().column(),
221     });
222
223     character = source.read();
224 }
225
226 const std::vector<Token>& Tokenizer::tokens() const {
227     return _tokens;
228 }
229
230 void Tokenizer::addToken(const Token& token) {
231     _tokens.push_back(token);
232 }
233
234 void Tokenizer::addError(const Error& error) {
235     _logger.message(error);
236 }

```

src\tokenizer\view\IdentifiersTableView\IdentifiersTableView.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <utility>
5 #include <vector>
6
7 #include <TableView.hpp>
8
9 using SymbolEntry = std::pair<std::string, size_t>;
10
11 class IdentifiersTableView {
12 private:
13     TableView<SymbolEntry> _table;
14
15 public:
16     IdentifiersTableView(
17         std::string title,
18         const std::vector<SymbolEntry>& data
19     );
20
21     void print() const;
22 };
```

```
1 #include "IdentifiersTableView.hpp"
2
3 IdentifiersTableView::IdentifiersTableView(
4     std::string title,
5     const std::vector<SymbolEntry>& data
6 ) :
7     _table(std::move(title), data) {
8     _table
9         .addColumn(
10             "Code", "{:>5}",
11             [](const auto& pair) { return std::to_string(pair.second); }
12         )
13         .addColumn(
14             "Value", "{:<30}",
15             [](const auto& pair) { return pair.first; }
16         );
17 }
18
19 void IdentifiersTableView::print() const {
20     _table.print();
21 }
```

```
1 #pragma once
2
3 #include <string>
4 #include <utility>
5 #include <vector>
6
7 #include <TableView.hpp>
8
9 using SymbolEntry = std::pair<std::string, size_t>;
10
11 class LiteralsTableView {
12 private:
13     TableView<SymbolEntry> _table;
14
15 public:
16     LiteralsTableView(
17         std::string title,
18         const std::vector<SymbolEntry>& data
19     );
20
21     void print() const;
22 };
```

```
1 #include "LiteralsTableView.hpp"
2
3 LiteralsTableView::LiteralsTableView(
4     std::string title,
5     const std::vector<SymbolEntry>& data
6 ) :
7     _table(std::move(title), data) {
8     _table
9         .addColumn(
10             "Code", "{:>5}",
11             [](const auto& pair) { return std::to_string(pair.second); }
12         )
13         .addColumn(
14             "Value", "{:<30}",
15             [](const auto& pair) { return pair.first; }
16         );
17 }
18
19 void LiteralsTableView::print() const {
20     _table.print();
21 }
```

src\tokenizer\view\TokensTableView\TokensTableView.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <vector>
5
6 #include <SymbolStore.hpp>
7 #include <TableView.hpp>
8 #include <Tokenizer.hpp>
9
10 class TokensTableView {
11 private:
12     TableView<Token> _table;
13
14 public:
15     TokensTableView(
16         std::string title,
17         const SymbolStore& symbols,
18         const std::vector<Token>& data
19     );
20
21     void print() const;
22 };
```

```
1 #include "TokensTableView.hpp"
2
3 #include <format>
4
5 std::string getTypeLabel(SymbolType type) {
6     switch (type) {
7         case SymbolType::Ascii:
8             return "Delimiter";
9         case SymbolType::MultiDelimiter:
10            return "Multi-Delimiter";
11        case SymbolType::Keyword:
12            return "Keyword";
13        case SymbolType::Literal:
14            return "Literal";
15        case SymbolType::Identifier:
16            return "Identifier";
17        default:
18            return "N/A";
19    }
20 }
21
22 TokensTableView::TokensTableView(
23     std::string title,
24     const SymbolStore& symbols,
25     const std::vector<Token>& data
26 ) :
27     _table(std::move(title), data) {
28     _table
29         .addColumn(
30             "Code", "{:>5}",
31             [](const auto& t) { return std::to_string(t.code); }
32         )
33         .addColumn(
34             "Row", "{:>3}",
35             [](const auto& t) { return std::to_string(t.row + 1); }
36         )
37         .addColumn(
38             "Col", "{:>3}",
39             [](const auto& t) { return std::to_string(t.column + 1); }
40         )
41         .addColumn(
42             "Type", "{:<15}",
43             [&symbols](const auto& t) {
44                 return getTypeLabel(symbols.lookupType(t.code));
45             }
46         )
47         .addColumn(
48             "Value", "{:<30}",
49             [&symbols](const auto& t) {
50                 return std::string(symbols.lookup(t.code));
51             }
52         );
53 }
54
55 void TokensTableView::print() const {
56     _table.print();
57 }
```

src\ui\ui.constants.hpp

```
1 #pragma once
2
3 namespace Box {
4 constexpr auto Horizontal = "—";
5 constexpr auto Vertical = "|";
6 constexpr auto DownAndRight = "┐";
7 constexpr auto DownAndLeft = "└";
8 constexpr auto UpAndRight = "┌";
9 constexpr auto UpAndLeft = "┘";
10 constexpr auto VerticalAndRight = "┤";
11 constexpr auto VerticalAndLeft = "├";
12 constexpr auto DownAndHorizontal = "┴";
13 constexpr auto UpAndHorizontal = "┬";
14 constexpr auto VerticalAndHorizontal = "┼";
15 } // namespace Box
```

src\ui\TableView\TableView.hpp

```
1  #pragma once
2
3  #include <format>
4  #include <functional>
5  #include <iostream>
6  #include <string>
7  #include <vector>
8
9  #include <ui.constants.hpp>
10
11 template <typename T>
12 class TableView {
13 private:
14     struct Column {
15         std::string name;
16         std::string format;
17         std::function<std::string(const T&)> extractor;
18     };
19
20     std::string _title;
21     std::vector<T> _data;
22     std::vector<Column> _columns;
23     std::ostream& _out;
24
25     [[nodiscard]] std::string border(
26         const std::string& left,
27         const std::string& junction,
28         const std::string& right
29     ) const;
30     [[nodiscard]] std::string header() const;
31     [[nodiscard]] std::string row(const T& item) const;
32     [[nodiscard]] size_t columnWidth(const Column& col) const;
33     static size_t parseWidth(const std::string& fmt);
34
35 public:
36     TableView(
37         std::string title,
38         std::vector<T> data,
39         std::ostream& out = std::cout
40     );
41
42     TableView& addColumn(
43         const std::string& colName,
44         const std::string& format,
45         std::function<std::string(const T&)> extractor
46     );
47
48     void print() const;
49 };
50
51 #include "TableView.tpp"
```

```
1  template <typename T>
2  TableView<T>::TableView(
3      std::string title,
4      std::vector<T> data,
5      std::ostream& out
6  ) :
7      _title(std::move(title)),
8      _data(std::move(data)),
9      _out(out) {}
10
11 template <typename T>
12 TableView<T>& TableView<T>::addColumn(
13     const std::string& colName,
14     const std::string& format,
15     std::function<std::string(const T&)> extractor
16 ) {
17     _columns.push_back({colName, format, std::move(extractor)});
18
19     return *this;
20 }
21
22 template <typename T>
23 void TableView<T>::print() const {
24     if (!_title.empty()) {
25         _out << _title << "\n";
26     }
27
28     _out << border(Box::DownAndRight, Box::DownAndHorizontal, Box::DownAndLeft) << "\n";
29     _out << header() << "\n";
30     _out << border(Box::VerticalAndRight, Box::VerticalAndHorizontal, Box::VerticalAndLeft) << "\n";
31
32     for (const auto& item : _data) {
33         _out << row(item) << "\n";
34     }
35
36     _out << border(Box::UpAndRight, Box::UpAndHorizontal, Box::UpAndLeft) << "\n";
37 }
38
39 template <typename T>
40 std::string TableView<T>::border(
41     const std::string& left,
42     const std::string& junction,
43     const std::string& right
44 ) const {
45     std::string result = left;
46
47     for (size_t i = 0; i < _columns.size(); ++i) {
48         size_t width = columnWidth(_columns[i]);
49
50         for (size_t j = 0; j < width + 2; ++j) {
51             result += Box::Horizontal;
52         }
53
54         result += (i < _columns.size() - 1) ? junction : right;
55     }
56
57     return result;
58 }
59
60 template <typename T>
61 std::string TableView<T>::header() const {
62     std::string result;
```

```

63     result += Box::Vertical;
64
65     for (const auto& column : _columns) {
66         result += std::format(" {:<{}} ", column.name, columnWidth(column));
67         result += Box::Vertical;
68     }
69
70     return result;
71 }
72
73 template <typename T>
74 std::string TableView<T>::row(const T& item) const {
75     std::string result;
76     result += Box::Vertical;
77
78     for (const auto& column : _columns) {
79         std::string value = column.extractor(item);
80         result += " " +
81             std::vformat(column.format, std::make_format_args(value)) +
82             " ";
83         result += Box::Vertical;
84     }
85
86     return result;
87 }
88
89 template <typename T>
90 size_t TableView<T>::columnWidth(const Column& column) const {
91     return std::max(parseWidth(column.format), column.name.length());
92 }
93
94 template <typename T>
95 size_t TableView<T>::parseWidth(const std::string& format) {
96     size_t width = 0;
97
98     for (char character : format) {
99         if (character >= '0' && character <= '9') {
100             width = (width * 10) + (character - '0');
101         }
102     }
103
104     return width;
105 }

```

src\ui\TreeView\TreeView.hpp

```
1  #pragma once
2
3  #include <functional>
4  #include <iostream>
5  #include <string>
6
7  #include <ui.constants.hpp>
8  #include <Tree.hpp>
9
10 template <typename T>
11 class TreeView {
12 private:
13     std::string _title;
14     const Tree<T>& _tree;
15     std::ostream& _out;
16     std::function<std::string(const T&)> _nodeFormatter;
17     std::function<std::string(const T&)> _edgeFormatter;
18
19     void printNode(
20         const TreeNode<T>& node,
21         const std::string& prefix,
22         bool isLast
23     ) const;
24
25 public:
26     TreeView(std::string title, const Tree<T>& tree, std::ostream& out = std::cout);
27
28     TreeView& setNodeFormatter(std::function<std::string(const T&)> formatter);
29     TreeView& setEdgeFormatter(std::function<std::string(const T&)> formatter);
30
31     void print() const;
32 };
33
34 #include "TreeView.cpp"
```

```
1  template <typename T>
2  TreeView<T>::TreeView(std::string title, const Tree<T>& tree, std::ostream& out) :
3      _title(std::move(title)),
4      _tree(tree),
5      _out(out),
6      _nodeFormatter(nullptr),
7      _edgeFormatter(nullptr) {}
8
9  template <typename T>
10 TreeView<T>& TreeView<T>::setNodeFormatter(
11     std::function<std::string(const T&)> formatter
12 ) {
13     _nodeFormatter = std::move(formatter);
14
15     return *this;
16 }
17
18 template <typename T>
19 TreeView<T>& TreeView<T>::setEdgeFormatter(
20     std::function<std::string(const T&)> formatter
21 ) {
22     _edgeFormatter = std::move(formatter);
23
24     return *this;
25 }
26
27 template <typename T>
28 void TreeView<T>::print() const {
29     auto& root = _tree.root();
30
31     _out << _title << "\n";
32     _out << _nodeFormatter(root.data()) << "\n";
33
34     for (const auto& child : root.children()) {
35         printNode(*child, "", child == root.children().back());
36     }
37 }
38
39 template <typename T>
40 void TreeView<T>::printNode(
41     const TreeNode<T>& node,
42     const std::string& prefix,
43     bool isLast
44 ) const {
45     std::string source = isLast
46         ? Box::UpAndRight
47         : Box::VerticalAndRight;
48
49     std::string horizontalConnector = Box::Horizontal;
50
51     std::string edgeLabel = _edgeFormatter(node.data());
52
53     std::string nodeLabel = _nodeFormatter(node.data());
54
55     std::string verticalConnector = isLast
56         ? " "
57         : Box::Vertical;
58
59     _out << prefix << source << horizontalConnector;
60
61     _out << edgeLabel << horizontalConnector;
62 }
```

```
63     _out << nodeLabel << "\n";
64
65     std::string childPrefix = prefix + verticalConnector + std::string(edgeLabel.size() + 2, ' ');
66
67     for (const auto& child : node.children()) {
68         printNode(*child, childPrefix, child == node.children().back());
69     }
70 }
```

src\utility\Args\Args.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <unordered_map>
5 #include <vector>
6
7 class Args {
8 private:
9     struct StringArg {
10         std::string shortName;
11         bool required;
12         std::string value;
13         bool provided = false;
14     };
15
16     struct FlagArg {
17         std::string shortName;
18         bool value = false;
19     };
20
21     std::string _program;
22     std::vector<std::string> _argv;
23
24     std::unordered_map<std::string, StringArg> _strings;
25     std::unordered_map<std::string, FlagArg> _flags;
26     std::unordered_map<std::string, std::string> _aliases;
27
28     void printUsage() const;
29
30 public:
31     Args(int argc, char* argv[]);
32
33     Args& expectString(
34         const std::string& name,
35         const std::string& shortName,
36         bool required = false
37     );
38     Args& expectFlag(
39         const std::string& name,
40         const std::string& shortName
41     );
42
43     Args& parse();
44
45     [[nodiscard]] const std::string& getString(const std::string& name) const;
46     [[nodiscard]] bool getFlag(const std::string& name) const;
47 };
```

```
1 #include "Args.hpp"
2
3 #include <format>
4 #include <iostream>
5 #include <stdexcept>
6
7 Args::Args(int argc, char* argv[]) : _program(argv[0]) {
8     for (int i = 1; i < argc; i++) {
9         _argv.emplace_back(argv[i]);
10    }
11 }
12
13 Args& Args::expectString(
14     const std::string& name,
15     const std::string& shortName,
16     bool required
17 ) {
18     _strings[name] = {shortName, required, "", false};
19     _aliases["--" + name] = name;
20     _aliases["-" + shortName] = name;
21     return *this;
22 }
23
24 Args& Args::expectFlag(
25     const std::string& name,
26     const std::string& shortName
27 ) {
28     _flags[name] = {shortName, false};
29     _aliases["--" + name] = name;
30     _aliases["-" + shortName] = name;
31     return *this;
32 }
33
34 Args& Args::parse() {
35     for (size_t i = 0; i < _argv.size(); i++) {
36         const auto& arg = _argv[i];
37
38         auto alias = _aliases.find(arg);
39
40         if (alias == _aliases.end()) {
41             std::cerr << "Unknown argument: " << arg << "\n";
42             printUsage();
43             exit(1);
44         }
45
46         const auto& name = alias->second;
47
48         auto stringArg = _strings.find(name);
49         if (stringArg != _strings.end()) {
50             if (i + 1 >= _argv.size()) {
51                 std::cerr << "Missing value for: " << arg << "\n";
52                 printUsage();
53                 exit(1);
54             }
55
56             stringArg->second.value = _argv[++i];
57             stringArg->second.provided = true;
58             continue;
59         }
60
61         auto flagArg = _flags.find(name);
62         if (flagArg != _flags.end()) {
```

```

63         flagArg->second.value = true;
64     }
65 }
66
67 for (const auto& [name, arg] : _strings) {
68     if (arg.required && !arg.provided) {
69         std::cerr << "Missing required argument: --" << name << "\n";
70         printUsage();
71         exit(1);
72     }
73 }
74
75 return *this;
76 }
77
78 void Args::printUsage() const {
79     std::cerr << "\nUsage: " << _program;
80
81     for (const auto& [name, arg] : _strings) {
82         if (arg.required) {
83             std::cerr << " --" << name << " <value>";
84         }
85     }
86
87     std::cerr << " [options]\n\nOptions:\n";
88
89     for (const auto& [name, arg] : _strings) {
90         std::cerr << std::format(
91             "  --{ }, -{ } <value>{ }\n", name, arg.shortName,
92             arg.required ? " (required)" : ""
93         );
94     }
95
96     for (const auto& [name, arg] : _flags) {
97         std::cerr << std::format("  --{ }, -{ }\n", name, arg.shortName);
98     }
99 }
100
101 const std::string& Args::getString(const std::string& name) const {
102     auto it = _strings.find(name);
103
104     if (it == _strings.end()) {
105         throw std::runtime_error("Unknown string argument: " + name);
106     }
107
108     return it->second.value;
109 }
110
111 bool Args::getFlag(const std::string& name) const {
112     auto it = _flags.find(name);
113
114     if (it == _flags.end()) {
115         throw std::runtime_error("Unknown flag argument: " + name);
116     }
117
118     return it->second.value;
119 }

```

```
1 #pragma once
2
3 #include <functional>
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 template <typename T>
9 class Logger {
10 private:
11     std::string _name;
12     std::function<std::string(const T&)> _formatter;
13     std::ostream& _out;
14     std::vector<T> _messages;
15
16 public:
17     Logger(
18         std::string name,
19         std::function<std::string(const T&)> formatter,
20         std::ostream& out = std::cout
21     );
22
23     void message(const T& data);
24
25     [[nodiscard]] const std::vector<T>& messages() const;
26     void clear();
27 };
28
29 #include "Logger.hpp"
```

```
1  template <typename T>
2  Logger<T>::Logger(
3      std::string name,
4      std::function<std::string(const T&)> formatter,
5      std::ostream& out
6  ) :
7      _name(std::move(name)), _formatter(std::move(formatter)), _out(out) {}
8
9  template <typename T>
10 void Logger<T>::message(const T& data) {
11     _messages.push_back(data);
12     _out << _name << " | " << _formatter(data) << "\n";
13 }
14
15 template <typename T>
16 const std::vector<T>& Logger<T>::messages() const {
17     return _messages;
18 }
19
20 template <typename T>
21 void Logger<T>::clear() {
22     _messages.clear();
23 }
```

```
1 #pragma once
2
3 #include <deque>
4 #include <functional>
5 #include <optional>
6 #include <vector>
7
8 template <typename T>
9 class Queue {
10 private:
11     std::deque<T> _items;
12
13 public:
14     Queue();
15     Queue(const std::vector<T>& items);
16
17     void push(const T& item);
18
19     std::optional<T> pop();
20     std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);
21
22     void clear();
23
24     std::optional<T> peek() const;
25     bool isEmpty() const;
26     size_t size() const;
27 };
28
29 #include "Queue.tpp"
```



```

1  template <typename T>
2  Queue<T>::Queue() : _items() {}
3
4  template <typename T>
5  Queue<T>::Queue(const std::vector<T>& items) :
6      _items(items.begin(), items.end()) {}
7
8  template <typename T>
9  void Queue<T>::push(const T& item) {
10     _items.push_back(item);
11 }
12
13 template <typename T>
14 std::optional<T> Queue<T>::pop() {
15     if (_items.empty()) {
16         return std::nullopt;
17     }
18
19     T item = std::move(_items.front());
20     _items.pop_front();
21
22     return item;
23 }
24
25 template <typename T>
26 std::optional<T> Queue<T>::popUntil(
27     const std::function<bool(const T&)>& predicate
28 ) {
29     while (!_items.empty()) {
30         if (predicate(_items.front())) {
31             return _items.front();
32         }
33
34         _items.pop_front();
35     }
36
37     return std::nullopt;
38 }
39
40 template <typename T>
41 void Queue<T>::clear() {
42     _items.clear();
43 }
44
45 template <typename T>
46 std::optional<T> Queue<T>::peek() const {
47     if (_items.empty()) {
48         return std::nullopt;
49     }
50
51     return _items.front();
52 }
53
54 template <typename T>
55 bool Queue<T>::isEmpty() const {
56     return _items.empty();
57 }
58
59 template <typename T>
60 size_t Queue<T>::size() const {
61     return _items.size();
62 }

```

```
1 #pragma once
2
3 #include <vector>
4 #include <optional>
5 #include <functional>
6
7 template <typename T>
8 class Stack {
9 private:
10     std::vector<T> _items;
11
12 public:
13     Stack();
14     Stack(const std::vector<T>& items);
15
16     void push(const T& item);
17
18     std::optional<T> pop();
19     std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);
20
21     void clear();
22
23     std::optional<T> peek() const;
24     bool isEmpty() const;
25     size_t size() const;
26 };
27
28 #include "Stack.cpp"
```

```
1  template <typename T>
2  Stack<T>::Stack() : _items() {}
3
4  template <typename T>
5  Stack<T>::Stack(const std::vector<T>& items) : _items(items) {}
6
7  template <typename T>
8  void Stack<T>::push(const T& item) {
9      _items.push_back(item);
10 }
11
12 template <typename T>
13 std::optional<T> Stack<T>::pop() {
14     if (_items.empty()) {
15         return std::nullopt;
16     }
17
18     T item = std::move(_items.back());
19     _items.pop_back();
20
21     return item;
22 }
23
24 template <typename T>
25 std::optional<T> Stack<T>::popUntil(const std::function<bool(const T&)>& predicate) {
26     while (!_items.empty()) {
27         if (predicate(_items.back())) {
28             return _items.back();
29         }
30
31         _items.pop_back();
32     }
33
34     return std::nullopt;
35 }
36
37 template <typename T>
38 void Stack<T>::clear() {
39     _items.clear();
40 }
41
42 template <typename T>
43 std::optional<T> Stack<T>::peek() const {
44     if (_items.empty()) {
45         return std::nullopt;
46     }
47
48     return _items.back();
49 }
50
51 template <typename T>
52 bool Stack<T>::isEmpty() const {
53     return _items.empty();
54 }
55
56 template <typename T>
57 size_t Stack<T>::size() const {
58     return _items.size();
59 }
```

src\utility\String\String.hpp

```
1 #pragma once
2
3 #include <string>
4
5 namespace String {
6     std::string repeat(const std::string& str, unsigned int times);
7 }
```

```
1 #include "String.hpp"
2
3 namespace String {
4
5 std::string repeat(const std::string& str, unsigned int times) {
6     std::string result = "";
7
8     for (unsigned int i = 0; i < times; i++) {
9         result += str;
10    }
11
12    return result;
13 }
14
15 }
```

src\utility\Tree\Tree.hpp

```
1 #pragma once
2
3 #include <TreeNode.hpp>
4
5 template <typename T>
6 class Tree {
7 private:
8     std::unique_ptr<TreeNode<T>> _root;
9
10 public:
11     Tree(T data);
12
13     TreeNode<T>& root();
14     const TreeNode<T>& root() const;
15 };
16
17 #include "Tree.cpp"
```

```
1  template <typename T>
2  Tree<T>::Tree(T data) : _root(std::make_unique<TreeNode<T>>(data)) {}
3
4  template <typename T>
5  TreeNode<T>& Tree<T>::root() {
6      return *_root;
7  }
8
9  template <typename T>
10 const TreeNode<T>& Tree<T>::root() const {
11     return *_root;
12 }
```

src\utility\Tree\Node\TreeNode.hpp

```
1 #pragma once
2
3 #include <vector>
4 #include <memory>
5
6 template <typename T>
7 class TreeNode {
8 private:
9     T _data;
10     std::vector<std::unique_ptr<TreeNode<T>>> _children;
11
12 public:
13     TreeNode(T data);
14     TreeNode(const TreeNode&) = delete;
15     TreeNode& operator=(const TreeNode&) = delete;
16     TreeNode(TreeNode&&) = default;
17     TreeNode& operator=(TreeNode&&) = default;
18
19     const T& data() const;
20     T& data();
21
22     const std::vector<std::unique_ptr<TreeNode<T>>>& children() const;
23     TreeNode<T>& grow(T data);
24     void prune();
25 };
26
27 #include "TreeNode.cpp"
```

```
1  template <typename T>
2  TreeNode<T>::TreeNode(T data) : _data(data) {}
3
4  template <typename T>
5  const T& TreeNode<T>::data() const {
6      return _data;
7  }
8
9  template <typename T>
10 T& TreeNode<T>::data() {
11     return _data;
12 }
13
14 template <typename T>
15 const std::vector<std::unique_ptr<TreeNode<T>>>& TreeNode<T>::children() const {
16     return _children;
17 }
18
19 template <typename T>
20 TreeNode<T>& TreeNode<T>::grow(T data) {
21     _children.push_back(std::make_unique<TreeNode<T>>(data));
22     return *_children.back();
23 }
24
25 template <typename T>
26 void TreeNode<T>::prune() {
27     _children.clear();
28 }
```